App Dev II Project Report
Hospital Managment System

1. Student Details

Name : Sumaiya Afroz
Roll : 24f1000754
Email : 24f1000754@ds.study.iitm.ac.in

About Me:
I am a student with a strong interest in full-stack web application development.

2. Project Details

Project Title: Quantified Self App

Problem Statement: To design and develop a web-based Hospital Management System that enables patients,doctors,admin.

Approach:

The application is developed using a modular full-stack architecture. Flask is used to build REST APIs for authentication, appointment handling, and role-based dashboards. The frontend is developed using Vue.js to provide dynamic dashboards for Admin, Doctor, and Patient roles. SQLite is used as the database along with SQLAlchemy ORM.

3. AI/LLM Declaration:

I used AI tools (ChatGPT 5) in a limited manner to:

- Get suggestions for structuring API endpoints
- Improve naming consistency in code
- Clarify error handling and debugging approaches

The estimated AI assistance is approx 27%, mainly for guidance,ideas during frontend development to make my landpage more attractice and some minor improvements.Used chat gpt for frontend UI development and styling suggestions
All major logic design, backend implementation, frontend integration, debugging, and testing were performed manually.

4. Technologies and Frameworks Used:

This project is developed using a full-stack approach with modern backend and frontend technologies.

Backend

- **Flask (Python):** Used to build REST APIs for authentication, appointments, admin, doctor, and patient modules.
- **SQLAlchemy & SQLite:** Used for database design and data handling.
- **Werkzeug Security:** Used for password hashing and authentication.

Background Jobs & Performance

- **Redis:** Used for caching frequently accessed data and improving API performance.
- **Celery:** Used for asynchronous background jobs such as reminders, report generation, and data export.
- **Scheduled Jobs:** Designed for daily appointment reminders and monthly activity reports.

Frontend

- **Vue.js**: Used to build dynamic dashboards for Admin, Doctor, and Patient.
- **Bootstrap:** Used for responsive UI design.
- **Axios:** Used for frontend-backend communication.

5.Database Schema / ER Diagram:

Tables
Users→ id, name, email, password_hash, role, is_active, created_at
Doctor→ id, user_id, specialization, bio, is_approved
Patient→ id, user_id, phone, gender, address, age
Appointment→ id, doctor_id, patient_id, date, time, status, diagnosis, prescription, created_at
Doctor Availibilty→ id, doctor_id, date, time, is_booked

Relationships
One User → One Doctor
One User → One Patient
One Doctor → Many Appointments
One Patient → Many Appointments
One Doctor → Many Availability Slots

6. API Resource Endpoints

Endpoint	Method	Description
/api/register	POST	Register new patient
/api/login	POST	Login user (admin/doctor/patient)
/api/admin/stats	GET	Get total doctors, patients, appointments
/api/admin/create-doctor	POST	Admin creates doctor profile
/api/patient/book-appointment	POST	Patient books appointment
/api/patient/appointments/<id>	GET	Patient appointment list
/api/doctor/appointments/<id>	GET	Doctor appointment list
/api/doctor/update-status	PUT	Update appointment status
/api/doctor/add-treatment	PUT	Add diagnosis & prescription

7. Architecture Overview:

- app.py – Main Flask application
- models.py – Database models (User, Doctor, Patient, Appointment)
- auth.py – Login & registration APIs
- admin.py – Admin related APIs
- doctor.py – Doctor related APIs
- patient.py – Patient related APIs
- Frontend built using Vue.js + Bootstrap
- Database: SQLite / MySQL with SQLAlchemy ORM

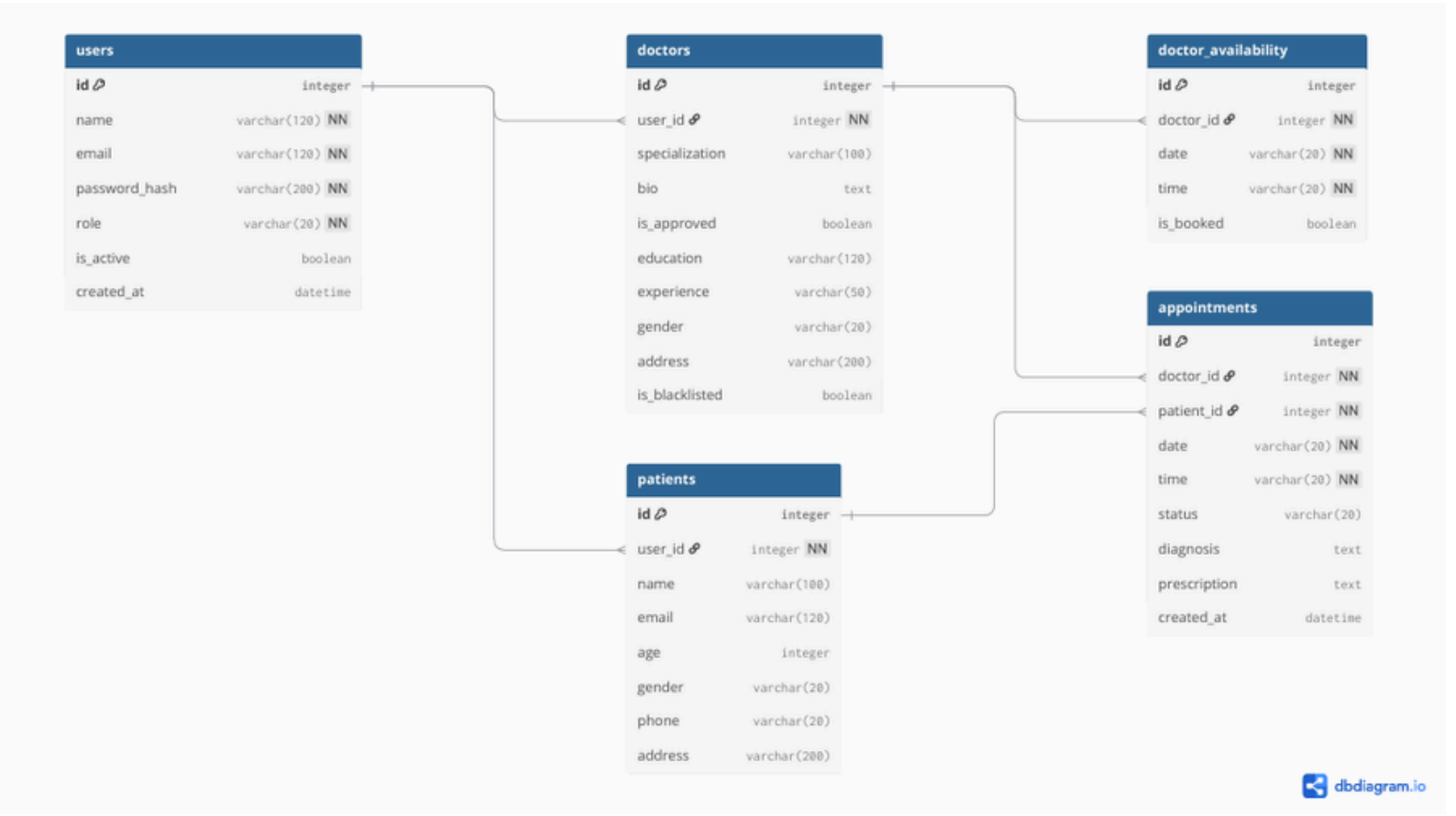
Implemented Features

- User authentication (Admin / Doctor / Patient)
- Patient registration and profile management
- Doctor creation and availability management
- Online appointment booking system
- Appointment status update (Booked, Completed, Cancelled)
- Doctor adds diagnosis and prescription
- Separate dashboards for Admin, Doctor, and Patient
- Caching used to improve performance

Additional Features

- Search doctors and patients
- Export patient treatment records
- Monthly and daily reports (backend jobs)
- Secure password hashing and role-based access

DB DIAGRAM:



Drive link :Demo video(project)